Contents

Adaptive LL Policy

Overview

Adaptive LL is a variant of the `least_loaded` policy that guarantees no empty labs whenever S ≥ F (students ≥ faculty). It does this by iteratively adjusting the tier-cap windows (N_A, N_B) during Phase 0 so that the number of empty labs after processing Tiers A and B is at most equal to the number of Class-C students. The core LL assignment rule is unchanged.

Pipeline

```
Phase 0a  →  Phase 0b (cap optimization)  →  Round 1  →  Main Allocation (LL rule)
```

Phase 0a — Tier Classification (identical to `least_loaded`)

- CPI percentile/quartile tiering → assign A / B / C membership
- Assign **baseline** N_tier caps: N_A = 3 (4 if S/F > 4), N_B = 5 (6 if S/F > 4)
- Compute max_load

Phase 0b — Cap Optimization

After Phase 0a, a cheap dry-run (`simulate_tiers_ab`) simulates Round 1 and Tiers A+B using the LL rule to count the number of faculty with zero students after those tiers (`E_after_B`).

`|C_remaining|` definition:

`|C_remaining|` = the number of Class-C students who were **not** assigned during the simulated Round 1. In most cohorts this equals `|C|` (because Round 1 only assigns one student per advisor and Class-C students rarely list a popular advisor as their first choice). It can be less than `|C|` if some C-tier students happened to win Round 1 picks.

Condition checked: | E_after_B vs |C_remaining| | Meaning | |—————————|———| | E_after_B $\leq$ |C_remaining| | No optimization needed. All empty labs will be filled by Class-C students. | | E_after_B > |C_remaining| | Empty labs guaranteed. Cap optimization runs. |

The excess — `E_after_B − |C_remaining|` when positive — is stored in `meta["E_baseline_excess"]` and shown in the Phase 0 modal.

Cap search loop:

```
N_A, N_B = baseline caps
loop:
    E = simulate_tiers_ab(students, faculty, N_A, N_B)
    if E ≤ |C_remaining|: done (converged)
    if N_B < F:  N_B += 1          # expand B first
    elif N_A < N_B: N_A += 1       # expand A only after B reaches F
    else:                          # N_A = N_B = F, still E > |C_remaining|
        → STRUCTURAL ISSUE
```

Invariant maintained throughout: N_A $\leq$ N_B $\leq$ F (A students are always at least as protected as B students).

Round 1 and Main Allocation

Identical to `least_loaded`, but using the optimized (or unchanged) N_A and N_B caps.

Warning Hierarchy

Both `least_loaded` and `adaptive_ll` show warnings when empty labs are detected or inevitable.

Level 0 — S < F pre-check (both policies)

Condition: S < F
Message: "Fewer students than faculty (S, F). At least F—S labs will be empty under any policy."
Options: Proceed anyway / Switch policy

Level 1 — E_after_B > |C| (both policies)

For **standard LL**: modal with "Proceed at risk" (empty labs will appear) and "Switch policy".

For **Adaptive LL**: Phase 0b optimization runs automatically. If caps converge, the modal shows the optimized caps and "Proceed with optimized caps". If a structural issue is found, only "Switch policy" is offered.

Structural Issues

A structural issue exists when N_A = N_B = F (full preference list for all tiers) still leaves E_after_B > |C|. This means:

- The cohort's preference distribution is so skewed that even with full-list access, the LL rule cannot distribute students across all labs.
- No window adjustment can fix this. Switching to `cpi_fill` is recommended.

The Phase 0 modal shows a "Structural deficit" badge when this occurs.

N_A $\leq$ N_B Invariant

The cap ordering invariant ensures that higher-CPI students (Tier A) always have at least as much preference protection as lower-CPI students (Tier B). The optimization expands N_B first (preserving the invariant) and only expands N_A after N_B has reached F.

Phase 0 Output

The **View Phase 0 data** modal shows baseline and optimized caps side by side (e.g., "N_B: 5 $\rightarrow$ 8 (optimized)") when caps were adjusted.

The **Save Phase 0 CSV** button exports `phase0_students.csv` with per-student tier and N_tier assignments (using the optimized caps). This file can be shared with students before the allocation proceeds.

Trade-offs vs Standard LL

Property	`least_loaded`	`adaptive_ll`
Empty-lab guarantee (S $\geq$ F)	No	Yes (unless structural)
Preference protection (N_tier)	Baseline caps	May expand caps
CPI-based protection ordering	Maintained	Maintained (N_A $\leq$ N_B)
Computation overhead	Phase 0 only	Phase 0 + cap search loop
Structural issue detection	No	Yes (flagged explicitly)

When baseline caps already satisfy E_after_B $\leq$ |C|, Adaptive LL produces an identical allocation to standard LL.

Key Functions (`allocation.py`)

Function	Role
`simulate_tiers_ab(students, faculty, N_A, N_B)`	Dry-run Round 1 + Tiers A+B; returns E_after_B. No snapshots, no mutations.
`check_empty_lab_risk(students, faculty, meta)`	Returns `("s_lt_f", count)` or `("e_gt_c", count)` or `None`.

Function	Role
`phase0_optimize_caps(students, faculty, meta)`	Cap search loop; returns `(N_A_opt, N_B_opt, E_after_B, structural)`.
`phase0(..., optimize=True)`	Calls optimize caps after classification; stores results in `meta`.